

# Code-Layout, Readability and Reusability

Phase 5 · Project Development Phase

Team ID: SWTID-2026-6305 · Niranjan S R (Team Lead) · Pavithran P R · Srinivas S M · Sriram S

This document explains how the setup automation and the Access Control script for the Script-Controlled ACL project are organised on the ServiceNow Personal Developer Instance dev443713.service-now.com, so that another administrator can read, re-run and reuse them without breaking the existing configuration. The project is native ServiceNow configuration, so the code here is the small set of server-side Background Scripts that build the environment plus the one Advanced ACL script that enforces read access.

## Organising principle

- Numbered background scripts 01 to 05, each run from System Definition > Scripts - Background, so the setup executes in a fixed, repeatable order.
- One concern per script: roles, test user, sample data, ACL wiring and verification are never mixed in the same file.
- Idempotent inserts: every script runs a GlideRecord query to check for an existing record before inserting, so re-running a script never creates duplicate roles, users or sample records.
- Header comments: each script opens with a short comment block stating its purpose, the table it touches, and whether security\_admin elevation is required first.
- The READ ACL logic lives in one standalone file, `acl_read_script.js`, instead of being retyped inside the ACL form, so the same tested code is pasted into the Advanced Script field.

## Script inventory

| Script file            | Purpose                                                                                                                                                                                           |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 01_create_roles.js     | Create the four custom roles bb1 (read), bb2 (create), bb3 (write), bb4 (delete) in sys_user_role only if they do not already exist.                                                              |
| 02_create_test_user.js | Create the 'EEE User' test account and grant it all four roles bb1 to bb4 through sys_user_has_role, skipping grants already present.                                                             |
| 03_seed_records.js     | Insert the four sample Institution Details records ECE001, EEE001, EEE002 and CSE001 with their Branch values, skipping any Student Roll Number already on the table.                             |
| 04_configure_acls.js   | Prepare the four record-level ACLs on u_institution_details and remove the auto-generated u_institution_details_user role so each ACL needs only its bb role; run after security_admin elevation. |
| 05_verify_setup.js     | Read-only checks that print role counts, record counts per branch and ACL wiring, used to confirm the environment before impersonation testing.                                                   |

| Script file        | Purpose                                                                                                                                  |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| acl_read_script.js | Standalone copy of the READ ACL Advanced script (admin OR bb1 returns true, else false) that is pasted into the Read ACL's Script field. |

## Readability and reuse choices

- Consistent naming: bb1 to bb4 map directly to read, create, write and delete, so the purpose of each role is clear at the ACL that requires it.
- The check-before-insert pattern is identical across scripts 01, 02 and 03, so a reader learns it once and recognises it everywhere.
- Verification is isolated in script 05 and never changes data, so it can be re-run at any time to confirm state.
- The standalone READ script is short and self-contained, so it is easy to review, test on its own in a Background Script, and reuse if another branch condition is added later.